

ASSIGNMENT-20.2

Roll no:2303A52074

Batch-37

Task 1 – Password Security Enhancement

Prompt

- 1 Generate a simple Python program for user registration and login.
- 2 Check this Python login code for security issues. If passwords are stored in plain text, fix it using hashing (bcrypt or hashlib) and give improved code. Improved user registration and login system with password hashing

Code

User registration

```
# User registration and login system
users = {}

def register():
    username = input("Enter a username: ")
    if username in users:
        print("Username already exists. Please choose a different username.")
        return
    password = input("Enter a password: ")
    users[username] = password
    print("Registration successful!")

def login():
    username = input("Enter your username: ")
    password = input("Enter your password: ")
    if username in users and users[username] == password:
        print("Login successful!")
    else:
        print("Invalid username or password.")

def main():
    while True:
        print("1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            register()
        elif choice == '2':
            login()
        elif choice == '3':
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Improved version

```

#Check this Python login code for security issues.
#If passwords are stored in plain text, fix it using hashing (bcrypt or hashlib) and give improved d
# Improved user registration and login system with password hashing
import bcrypt
users = {}
def register():
    username = input("Enter a username: ").strip()
    if not username:
        print("Username cannot be empty.")
        return
    if username in users:
        print("Username already exists.")
        return
    password = input("Enter a password: ").strip()
    if len(password) < 6:
        print("Password must be at least 6 characters.")
        return
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    users[username] = hashed_password
    print("Registration successful!")
def login():
    username = input("Enter your username: ").strip()
    password = input("Enter your password: ").strip()
    if username in users:
        if bcrypt.checkpw(password.encode('utf-8'), users[username]):
            print("Login successful!")
        else:
            print("Incorrect password.")
    else:
        print("Username not found.")
def main():
    while True:
        print("1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            register()
        elif choice == '2':
            login()
        elif choice == '3':
            print("Exiting program.")
            break
        else:
            print("Invalid choice.")
if __name__ == "__main__":
    main()

```

Output

```
1. Register
2. Login
3. Exit
Enter your choice: 1
Enter a username: 2074
Enter a password: 2303a52074
Registration successful!

1. Register
2. Login
3. Exit
Enter your choice: 2
Enter your username: 2074
Enter your password: 2303a52074
Login successful!

1. Register
2. Login
3. Exit
Enter your choice: 3
Exiting program.
```

Explanation

We first checked the login code and saw that storing passwords directly is unsafe. Then we improved it by converting passwords into secure hashed values using bcrypt. During login, we compare the hashed version instead of the actual password. This makes the system safer because even if data is leaked, real passwords are not exposed.

Task 2 – Secure API Key Management

Prompt

Generate a simple Python script that connects to an external API using the requests library.

Analyze the following Python code that uses an API key.

1. Check if the API key is hardcoded.
2. Explain the risks of hardcoding API keys.

Code

```
ai 18.3 > 1.py > ...
1 import os
2 import requests
3 def get_data():
4     # Get API key from environment
5     api_key = os.getenv("API_KEY")
6     if not api_key:
7         print("❌ API key not found. Please set API_KEY.")
8         return
9     print("✅ API key loaded successfully!")
10    # Using free API (no key required, just for demo)
11    url = "https://jsonplaceholder.typicode.com/posts/1"
12    try:
13        response = requests.get(url)
14        if response.status_code == 200:
15            data = response.json()
16            print("\n🌟 Data fetched successfully:")
17            print(data)
18            print("\n🔒 API Key is NOT exposed in code ✓")
19        else:
20            print("Error:", response.status_code)
21    except Exception as e:
22        print("Error occurred:", e)
23 if __name__ == "__main__":
24     get_data()
25
```

Output

```
✅ API key loaded successfully!
🌟 Data fetched successfully:
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipi
nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet
chitecto'}
🔒 API Key is NOT exposed in code ✓
```

Explanation

The original program stored the API key directly in the source code, which is insecure. If the code is shared or uploaded, the API key can be exposed and misused.

We improved the program by storing the API key in environment variables using `os.getenv()`.

This ensures the key is not visible in the code while still allowing the program to work correctly.

Task 3 – Weak Encryption Detection

Prompt

Generate a simple encryption-decryption program using a basic or outdated method (e.g., Caesar cipher).

Improved encryption-decryption program using Caesar cipher with better security practices

Check this Python encryption code for weak methods.

If found, replace it with strong encryption (Fernet) and give secure code.

Code

```

#generate a simple encryption-decryption program using a basic or outdated method (e.g., Caesar ci
#Improved encryption-decryption program using Caesar cipher with better security practices
def caesar_cipher_encrypt(plain_text, shift):
    encrypted_text = ""
    for char in plain_text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                encrypted_char = chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
            else:
                encrypted_char = chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
            encrypted_text += encrypted_char
        else:
            encrypted_text += char
    return encrypted_text
def caesar_cipher_decrypt(encrypted_text, shift):
    decrypted_text = ""
    for char in encrypted_text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                decrypted_char = chr((ord(char) - ord('a') - shift_amount) % 26 + ord('a'))
            else:
                decrypted_char = chr((ord(char) - ord('A') - shift_amount) % 26 + ord('A'))
            decrypted_text += decrypted_char
        else:
            decrypted_text += char
    return decrypted_text
if __name__ == "__main__":
    text = input("Enter the text to encrypt: ")
    shift = int(input("Enter the shift value: "))
    encrypted = caesar_cipher_encrypt(text, shift)
    print(f"Encrypted text: {encrypted}")
    #print(f"Encrypted text: {encrypted}")
    decrypted = caesar_cipher_decrypt(encrypted, shift)
    print(f"Decrypted text: {decrypted}")
#Check this Python encryption code for weak methods.
#If found, replace it with strong encryption (Fernet) and give secure code.
from cryptography.fernet import Fernet
def generate_key():
    return Fernet.generate_key()
def encrypt_message(message, key):
    fernet = Fernet(key)
    encrypted_message = fernet.encrypt(message.encode())
    return encrypted_message
def decrypt_message(encrypted_message, key):
    fernet = Fernet(key)
    decrypted_message = fernet.decrypt(encrypted_message).decode()
    return decrypted_message
if __name__ == "__main__":
    key = generate_key()
    print(f"Encryption Key: {key.decode()}")
    message = input("Enter the message to encrypt: ")
    encrypted = encrypt_message(message, key)
    print(f"Encrypted message: {encrypted.decode()}")
    decrypted = decrypt_message(encrypted, key)
    print(f"Decrypted message: {decrypted}")

```

Output

```

Enter the text to encrypt: RAM
Enter the shift value: 1
Encrypted text: SBN
Decrypted text: RAM
Encryption Key: vG66SYIbOj9CEfERuyyJ0H8Edxhno9jEXxw8Hmu_iCY=
Enter the message to encrypt: RAM
Encrypted message: gAAAAABpy0yJDct46_mjprdSGT7YFiRmAFfDcvHZZdhBUTGW_q68FtGN12QexA2Xk7W9ZBLkJrkbWfYPFW8kPu2_15iUfdx5A==
Decrypted message: RAM

```

Explanation

The original program used a Caesar cipher, which is a weak encryption method. It only shifts letters and does not properly secure data like numbers or passwords. So, it can be easily reversed without any key, making it insecure. We improved it by using strong encryption (Fernet), which uses a secret key. Now, the data is securely encrypted and cannot be accessed without the correct key.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service Prompt

1#Generate a simple Python Flask web service for user login.

2.#improve this Flask web service code for security issues. #Find at least 3 vulnerabilities and fix them with secure coding practices

Code

1

```
ai 20.2 > 4.py > ...
1  #Generate a simple Python Flask web service for user login.
2  # Improved Flask web service for user login with password hashing
3  from flask import Flask, request, jsonify
4  import bcrypt
5  app = Flask(__name__)
6  users = {}
7  @app.route('/register', methods=['POST'])
8  def register():
9      data = request.get_json()
10     username = data.get('username', '').strip()
11     password = data.get('password', '').strip()
12     if not username:
13         return jsonify({"error": "Username cannot be empty."}), 400
14     if username in users:
15         return jsonify({"error": "Username already exists."}), 400
16     if len(password) < 6:
17         return jsonify({"error": "Password must be at least 6 characters."}), 400
18     hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
19     users[username] = hashed_password
20     return jsonify({"message": "Registration successful!"}), 201
21 @app.route('/login', methods=['POST'])
22 def login():
23     data = request.get_json()
24     username = data.get('username', '').strip()
25     password = data.get('password', '').strip()
26     if username in users and bcrypt.checkpw(password.encode('utf-8'), users[username]):
27         return jsonify({"message": "Login successful!"}), 200
28     else:
29         return jsonify({"error": "Invalid username or password."}), 401
30 if __name__ == '__main__':
31     app.run(debug=False)
32
```

2.

```

ai 20.2 > 4.py > login
--
33 #improve this Flask web service code for security issues.
34 #Find at least 3 vulnerabilities and fix them with secure coding practices
35 from flask import Flask, request, jsonify
36 import bcrypt
37 import re
38 app = Flask(__name__)
39 users = {}
40 # Password strength check
41 def is_strong_password(password):
42     return (len(password) >= 6 and
43             re.search(r"[A-Z]", password) and
44             re.search(r"[a-z]", password) and
45             re.search(r"[0-9]", password))
46 @app.route('/register', methods=['POST'])
47 def register():
48     data = request.get_json()
49     if not data:
50         return jsonify({"error": "Invalid input"}), 400
51     username = data.get('username', '').strip()
52     password = data.get('password', '').strip()
53     if not username:
54         return jsonify({"error": "Username cannot be empty"}), 400
55     if username in users:
56         return jsonify({"error": "Username already exists"}), 400
57     if not is_strong_password(password):
58         return jsonify({
59             "error": "Password must contain uppercase, lowercase, number and be at least 6 characters"
60         }), 400
61     hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
62     users[username] = hashed_password
63     return jsonify({"message": "Registration successful"}), 201
64 @app.route('/login', methods=['POST'])
65 def login():
66     data = request.get_json()
67     if not data:
68         return jsonify({"error": "Invalid input"}), 400
69     username = data.get('username', '').strip()
70     password = data.get('password', '').strip()
71     if username not in users:
72         return jsonify({"error": "User not found"}), 404
73     if bcrypt.checkpw(password.encode('utf-8'), users[username]):
74         return jsonify({"message": "Login successful"}), 200
75     else:
76         return jsonify({"error": "Incorrect password"}), 401
77 @app.route('/')
78 def home():
79     return "Secure Flask API is running"
80 if __name__ == '__main__':
81     app.run(debug=False)

```

Output

```

C:\Users\gauti\OneDrive\Documents\SEM-6\AI\docs\codes\ai
* Serving Flask app '4'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
* Running on http://127.0.0.1:5000

```



Explanation

We improved the web service by adding strong input validation and password rules. Passwords are securely stored using bcrypt hashing instead of plain text. Error handling and request validation were added to prevent misuse. The final API is more secure and suitable for real-world applications.

Task 5 – Exception Handling Improvement

Prompt

#generate a simple program that takes user input and performs arithmetic operations.#Analyze the following Python program.
#Identify possible runtime errors (invalid input, division by zero).
#Analyze the following Python program.
#Identify possible runtime errors (invalid input, division by zero).

Code

ai 20.2 > 5.py > ...

```
1 #generate a simple program that takes user input and performs arithmetic operations.
2 # Improved program that takes user input and performs arithmetic operations with error handling
3 def get_number(prompt):
4     while True:
5         try:
6             return float(input(prompt))
7         except ValueError:
8             print("Invalid input. Please enter a number.")
9
10 def main():
11     num1 = get_number("Enter the first number: ")
12     num2 = get_number("Enter the second number: ")
13     operation = input("Enter the operation (+, -, *, /): ").strip()
14
15     if operation == '+':
16         result = num1 + num2
17     elif operation == '-':
18         result = num1 - num2
19     elif operation == '*':
20         result = num1 * num2
21     elif operation == '/':
22         if num2 == 0:
23             print("Error: Division by zero is not allowed.")
24             return
25         result = num1 / num2
26     else:
27         print("Invalid operation. Please choose from +, -, *, /.")
28         return
29
30     print(f"The result of {num1} {operation} {num2} is: {result}")
31
32 if __name__ == "__main__":
33     main()
```

2

```
9 def get_number(prompt):
10     while True:
11         try:
12             return float(input(prompt))
13         except ValueError:
14             print("Invalid input. Please enter a valid number.")
15
16 def main():
17     try:
18         num1 = get_number("Enter the first number: ")
19         num2 = get_number("Enter the second number: ")
20
21         operation = input("Enter the operation (+, -, *, /): ").strip()
22
23         if not operation:
24             print("Operation cannot be empty.")
25             return
26
27         if operation == '+':
28             result = num1 + num2
29
30         elif operation == '-':
31             result = num1 - num2
32
33         elif operation == '*':
34             result = num1 * num2
35
36         elif operation == '/':
37             if num2 == 0:
38                 print("Error: Division by zero is not allowed.")
39                 return
40             result = num1 / num2
```

```

elif operation == '/':
    if num2 == 0:
        print("Error: Division by zero is not allowed.")
        return
    result = num1 / num2

else:
    print("Invalid operation. Use +, -, *, / only.")
    return

print(f"Result: {result}")

except Exception as e:
    print("Unexpected error occurred:", e)

if __name__ == "__main__":
    main()

```

Output

```

Enter the first number: 12
Enter the second number: 22
Enter the operation (+, -, *, /): /
The result of 12.0 / 22.0 is: 0.5454545454545454
Enter the first number: 21
Enter the second number: 21
Enter the operation (+, -, *, /): -
Result: 0.0

```

Explanation

The original program could crash when the user enters invalid input or divides by zero. We improved it by adding try-except blocks to handle these errors. Now the program shows clear error messages instead of stopping suddenly. This makes the program more reliable and easier for users to use.